	Result	Time	Cycles	GPU	SM Frequency	Process	Attributes
Current	813 - scan_blocks (131072, 1, 1)x(512, 1, 1)	10.52 msecond	17,044,025	0 - NVIDIA GeForce RTX 4060 Laptop GPU	1.62 cycle/nsecond	[9712] prefix_sum_gpu	

GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	81.00	Duration [msecond]	10.52
Memory Throughput [%]	81.00	Elapsed Cycles [cycle]	17,044,025
L1/TEX Cache Throughput [%]	81.02	SM Active Cycles [cycle]	17,040,653.08
L2 Cache Throughput [%]	7.79	SM Frequency [cycle/nsecond]	1.62
DRAM Throughput [%]	39.40	DRAM Frequency [cycle/nsecond]	7.99

High Throughput The kernel is utilizing greater than 80.0% of the available compute or memory performance of the device. To further improve performance, work will likely need to be shifted from the most utilized to another unit. Start by analyzing workloads in the Compute Workload Analysis section.

Roofline Analysis The ratio of peak float (fp32) to double (fp64) performance on this device is 64:1. The kernel achieved close to 0% of this device's fp32 peak performance and 0% of its fp64 peak performance. See the Kernel Profiling Guide for more details on roofline analysis.

Floating Point Operations Roofline

PM Sampling

Timeline view of PM metrics sampled periodically over the workload duration. Data is collected across multiple passes. Use this section to understand how workload behavior changes over its runtime.

Maximum Sampling Interval [usecond]	512	# Pass Groups	2
Maximum Buffer Size [Mbytes]	3.88	Dropped Samples [sample]	0

Compute Workload Analysis

Detailed analysis of the compute resources of the streaming multiprocessors (SM), including the achieved instructions per clock (IPC) and the utilization of each available pipeline. Pipelines with very high utilization might limit the overall performance.

Executed Ipc Elapsed [inst/cycle]	0.92	SM Busy [%]	34.87
Executed Ipc Active [inst/cycle]	0.92	Issue Slots Busy [%]	22.95
Issued Ipc Active [inst/cycle]	0.92		

Low Utilization
Est. Local Speedup: 90.26%
All compute pipelines are under-utilized. Either this kernel is very small or it doesn't issue enough warps per scheduler. Check the Launch Statistics and Scheduler Statistics sections for further details.

Memory Workload Analysis

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/second]	100.77	Mem Busy [%]	76.20
L1/TEX Hit Rate [%]	0.10	Max Bandwidth [%]	81.00
L2 Hit Rate [%]	50.18	Mem Pipes Busy [%]	81.00
L2 Compression Success Rate [%]	0	L2 Compression Ratio	0

L1TEX Global Store Access Pattern
Est. Speedup: 0.55%
The memory access pattern for global stores to L1TEX might not be optimal. On average, only 31.8 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced global stores.

Shared Load Bank Conflicts
Est. Speedup: 61.41%
The memory access pattern for shared loads might not be optimal and causes on average a 5.1 - way bank conflict across all 19005440 shared load requests.This results in 73695380 bank conflicts, which represent 75.80% of the overall 97225551 wavefronts for shared loads. Check the Source Counters section for uncoalesced shared loads.

Shared Store Bank Conflicts
Est. Speedup: 61.20%
The memory access pattern for shared stores might not be optimal and causes on average a 4.6 - way bank conflict across all 16384000 shared store requests.This results in 57061355 bank conflicts, which represent 75.54% of the overall 75542507 wavefronts for shared stores. Check the Source Counters section for uncoalesced shared stores.

Scheduler Statistics

Summary of the activity of the schedulers issuing instructions. Each scheduler maintains a pool of warps that it can issue instructions for. The upper bound of warps in the pool (Theoretical Warps) is limited by the launch configuration. On every cycle each scheduler checks the state of the allocated warps in the pool (Active Warps). Active warps that are not stalled (Eligible Warps) are ready to issue their next instruction. From the set of eligible warps the scheduler selects a single warp from which to issue one or more instructions (Issued Warp). On cycles with no eligible warps, the issue slot is skipped and no instruction is issued. Having many skipped issue slots indicates poor latency hiding.

Active Warps Per Scheduler [warp]	11.62	No Eligible [%]	77.05
Eligible Warps Per Scheduler [warp]	0.84	One or More Eligible [%]	22.95
Issued Warp Per Scheduler	0.23		

Issue Slot Utilization
Est. Local Speedup: 19.00%
Every scheduler is capable of issuing one instruction per cycle, but for this kernel each scheduler only issues an instruction every 4.4 cycles. This might leave hardware resources underutilized and may lead to less optimal performance. Out of the maximum of 12 warps per scheduler, this kernel allocates an average of 11.62 active warps per scheduler, but only an average of 0.84 warps were eligible per cycle. Eligible warps are the subset of active warps that are ready to issue their next instruction. Every cycle with no eligible warp results in no instruction being issued and the issue slot remains unused. To increase the number of eligible warps, avoid possible load imbalances due to highly different execution durations per warp. Reducing stalls indicated on the Warp State Statistics and Source Counters sections can help, too.

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	50.62	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	50.62	Avg. Not Predicated Off Threads Per Warp	18.05

Mio Throttle Stalls
Est. Speedup: 19.00%
On average, each warp of this kernel spends 16.9 cycles being stalled waiting for the MIO (memory input/output) instruction queue to be not full. This stall reason is high in cases of extreme utilization of the MIO pipelines, which include special math instructions, dynamic branches, as well as shared memory instructions. When caused by shared memory accesses, trying to use fewer but wider loads can reduce pipeline pressure. This stall type represents about 33.4% of the total average of 50.6 cycles between issuing two instructions.

Barrier Stalls
Est. Speedup: 19.00%
On average, each warp of this kernel spends 16.5 cycles being stalled waiting for sibling warps at a CTA barrier. A high number of warps waiting at a barrier is commonly caused by diverging code paths before a barrier. This causes some warps to wait a long time until other warps reach the synchronization point. Whenever possible, try to divide up the work into blocks of uniform workloads. If the block size is 512 threads or greater, consider splitting it into smaller groups. This can increase eligible warps without affecting occupancy, unless shared memory becomes a new occupancy limiter. Also, try to identify which barrier instruction causes the most stalls, and optimize the code executed before that synchronization point first. This stall type represents about 32.6% of the total average of 50.6 cycles between issuing two instructions.

Warp Stall Check the Warp Stall Sampling (All Samples) table for the top stall locations in your source based on sampling data. The Kernel Profiling Guide provides more details on each stall reason.

Thread Divergence
Est. Speedup: 35.30%
Instructions are executed in warps, which are groups of 32 threads. Optimal instruction throughput is achieved if all 32 threads of a warp execute the same instruction. The chosen launch configuration, early thread completion, and divergent flow control can significantly lower the number of active threads in a warp per cycle. This kernel achieves an average of 32.0 threads being active per cycle. This is further reduced to 18.1 threads per warp due to predication. The compiler may use predication to avoid an actual branch. Instead, all instructions are scheduled, but a per-thread condition code or predicate controls which threads execute the instructions. Try to avoid different execution paths within a warp when possible.

Instruction Statistics

Statistics of the executed low-level assembly instructions (SASS). The instruction mix provides insight into the types and frequency of the executed instructions. A narrow mix of instruction types implies a dependency on few instruction pipelines, while others remain unused. Using multiple pipelines allows hiding latencies and enables parallel execution. Note that 'Instructions/Opcode' and 'Executed Instructions' are measured differently and can diverge if cycles are spent in system calls.

Executed Instructions [inst]	375,390,208	Avg. Executed Instructions Per Scheduler [inst]	3,910,314.67
Issued Instructions [inst]	375,399,388	Avg. Issued Instructions Per Scheduler [inst]	3,910,410.29

FP32 Non-Fused Instructions
Est. Speedup: 2.37%
This kernel executes 0 fused and 41943040 non-fused FP32 instructions. By converting pairs of non-fused instructions to their fused, higher-throughput equivalent, the achieved FP32 performance could be increased by up to 50% (relative to its current performance). Check the Source page to identify where this kernel executes FP32 instructions.

NVLink Topology

NVLink Topology diagram shows logical NVLink connections with transmit/receive throughput.

NVLink Tables

Detailed tables with properties for each NVLink.

NUMA Affinity

Non-uniform memory access (NUMA) affinities based on compute and memory distances for all GPUs.

Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	131,072	Function Cache Configuration	CachePreferNone
Registers Per Thread [register/thread]	28	Static Shared Memory Per Block [Kbyte/block]	4.10
Block Size	512	Dynamic Shared Memory Per Block [byte/block]	0
Threads [thread]	67,108,864	Driver Shared Memory Per Block [Kbyte/block]	1.02
Waves Per SM	1,820.44	Shared Memory Configuration Size [Kbyte]	32.77
Uses Green Context	0	# SMs [SM]	24

Occupancy

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	4
Theoretical Active Warps per SM [warp]	48	Block Limit Shared Mem [block]	6
Achieved Occupancy [%]	96.99	Block Limit Warps [block]	3
Achieved Active Warps Per SM [warp]	46.55	Block Limit SM [block]	24

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	17,040,653.08	Average L1 Active Cycles [cycle]	17,040,653.08
Average L2 Active Cycles [cycle]	15,004,181.81	Average SMSP Active Cycles [cycle]	17,039,276.38
Average DRAM Active Cycles [cycle]	33,130,316	Total SM Elapsed Cycles [cycle]	409,056,464
Total L1 Elapsed Cycles [cycle]	409,056,464	Total L2 Elapsed Cycles [cycle]	267,654,000
Total SMSP Elapsed Cycles [cycle]	1,636,225,856	Total DRAM Elapsed Cycles [cycle]	336,377,856

Source Counters

Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.

Branch Instructions [inst]	4,194,304	Branch Efficiency [%]	0
Branch Instructions Ratio [%]	0.01	Avg. Divergent Branches	0

Uncoalesced Shared Accesses
Est. Speedup: 75.65%
This kernel has uncoalesced shared accesses resulting in a total of 129630208 excessive wavefronts (76% of the total 171311104 wavefronts). Check the L1 Wavefronts Shared Excessive table for the primary source locations. The CUDA Best Practices Guide has an example on optimizing shared memory accesses.

L1 Wavefronts Shared Excessive

Location	Value	Value (%)
0x764b07281ae0 in scan_blocks	4,063,232	3
0x764b07281ad0 in scan_blocks	4,063,232	3
0x764b07281ab0 in scan_blocks	4,063,232	3
0x764b07281aa0 in scan_blocks	4,063,232	3
0x764b072815f0 in scan_blocks	4,063,232	3

Follow the *rules outputs* to get guidance on how to navigate through the report and quickly discover performance bottlenecks in this kernel.
You could also disable individual sections to focus on selected performance aspects and make profiling faster.